==========

==========

CHAPTER 21 — MMU DESIGN PATTERNS FOR REAL SYSTEMS

==========

==========

A maximum-detail, RTOS/QNX-style breakdown of practical MMU layouts used in real operating systems, hypervisors, embedded devices, and safety-critical platforms.

This chapter shifts from theory → **real engineering patterns**. These are the designs you actually see in Linux, QNX, hypervisors, automotive ECUs, and embedded SoCs.

****

****

21.1 — Purpose of MMU Design Patterns

****

****

Real systems must balance:

- performance
- safety
- determinism
- security
- simplicity
- debuggability
- SMP scalability
- virtualization support

This chapter provides **canonical MMU layouts** used in production systems.

****

****

21.2 — Pattern A: Classic OS Kernel (Linux/QNX)

****

****

Characteristics

- TTBR1_EL1 → kernel space
- TTBR0_EL1 → user space
- global kernel mappings
- per-process user mappings
- ASID-tagged user space
- blocks for kernel
- pages for user space

Memory Layout

```
[High VA]  Kernel text (1 GB block)
[High VA]  Kernel data (1 GB block)
[High VA]  Kernel modules (2 MB blocks)
[High VA]  Kernel stacks (4 KB pages)
[High VA]  Device MMIO (4 KB pages, Device-nGnRnE)

[Low VA]   User code (4 KB pages)
[Low VA]   User data (4 KB pages)
[Low VA]   Heap (4 KB pages)
[Low VA]   Stack (4 KB pages)
```

Why this pattern works

- Kernel always mapped → fast syscall entry
- User space isolated via ASID
- Kernel uses blocks → minimal TLB pressure
- User uses pages → fine-grained protection

****

****

21.3 — Pattern B: Hypervisor (EL2)

****

****

Characteristics

- TTBR0_EL2 → Stage-2 tables
- VMID-tagged mappings

- blocks for guest RAM
- pages for device memory
- strict Device-nGnRnE for MMIO
- S2AP/S2PXN/S2UXN enforce security

Memory Layout

```
IPA → PA mapping:
    Guest RAM: 1 GB or 2 MB blocks
    Guest kernel: 2 MB blocks
    Guest user: 4 KB pages
    Guest MMIO: 4 KB pages (Device-nGnRnE)
```

Why this pattern works

- Stage-2 blocks reduce virtualization overhead
- Stage-2 pages allow fine-grained device isolation
- VMID ensures multi-VM isolation
- Hypervisor overrides guest attributes

****

****

21.4 — Pattern C: Bare-Metal Embedded System

****

****

Characteristics

- No user/kernel split
- Single TTBR
- Mostly identity-mapped
- MMIO mapped as Device-nGnRnE
- RAM mapped as Normal WB
- deterministic layout

Memory Layout

```
0x0000_0000 → 0x3FFF_FFFF   RAM (Normal WB, Inner Shareable)
0x4000_0000 → 0x4FFF_FFFF   MMIO (Device-nGnRnE)
0x5000_0000 → 0x5FFF_FFFF   Flash (Normal WT or WB)
```

Why this pattern works

- simplicity
- deterministic timing
- minimal TLB pressure
- safe device access

****

****

21.5 — Pattern D: Safety-Critical RTOS (QNX, AUTOSAR)

****

****

Characteristics

- strict partitioning
- static ASID ranges
- static VMID ranges
- deterministic TTBR1 kernel
- per-partition TTBR0
- no dynamic remapping
- no overcommit
- no fragmentation

Memory Layout

```
Partition 1 (Safety):
    Code: 2 MB block
    Data: 2 MB block
    MMIO: 4 KB pages

Partition 2 (Non-Safety):
    Code: 2 MB block
    Data: 2 MB block
    Heap: 4 KB pages
```

```
Kernel:
    Global mappings (1 GB blocks)
```

Why this pattern works

- deterministic behavior
- strict isolation
- minimal TLB invalidation
- predictable timing

****

****

21.6 — Pattern E: High-Performance Server

****

****

Characteristics

- huge pages (2 MB / 1 GB)
- NUMA-aware mappings
- per-core ASID pools
- aggressive TLB optimization
- minimal fragmentation
- Stage-2 for VM isolation

Memory Layout

```
Node 0:
    RAM: 1 GB blocks
Node 1:
```

```
    RAM: 1 GB blocks

Kernel:
    1 GB blocks

User:
    2 MB huge pages
```

Why this pattern works

- minimal TLB misses
- minimal page-table walks
- optimal cache locality
- NUMA-aware performance

****

****

21.7 — Pattern F: GPU/Accelerator-Heavy System

****

****

Characteristics

- IOMMU required
- device memory mapped separately
- DMA buffers mapped Normal WB
- MMIO mapped Device-nGnRnE
- shared memory regions for GPU

Memory Layout

```
CPU RAM: Normal WB
GPU RAM: Normal WB or Device-nGnRE (depending on hardware)
GPU MMIO: Device-nGnRnE
Shared buffers: Normal WB, Inner Shareable
```

Why this pattern works

- safe DMA
- high throughput
- correct device ordering

****

****

21.8 — Pattern G: Automotive ECU

****

****

Characteristics

- deterministic timing
- strict safety partitions
- static mappings
- no dynamic memory allocation
- MMIO isolation
- watchdog-safe layout

Memory Layout

```
Safety Partition:
    Code: 2 MB block
    Data: 2 MB block
    MMIO: Device-nGnRnE

Non-Safety Partition:
    Code: 2 MB block
    Data: 2 MB block

Kernel:
    Global mappings
```

Why this pattern works

- deterministic behavior
- strict isolation
- safety compliance

****

****

21.9 — Pattern H: Secure Enclave / TEE

****

****

Characteristics

- EL3 or Secure EL1
- secure RAM region

- secure MMIO region
- non-secure world isolated
- TrustZone enforcement

Memory Layout

```
Secure World:
    Secure RAM: Normal WB
    Secure MMIO: Device-nGnRnE

Non-Secure World:
    Normal RAM: Normal WB
    MMIO: Device-nGnRnE
```

Why this pattern works

- strong isolation
- secure memory attributes
- secure device access

****

****

21.10 — Pattern I: Virtualized Cloud Host

****

****

Characteristics

- heavy Stage-2 usage
- memory overcommit

- ballooning
- live migration
- shadow page tables
- paravirtualization

Memory Layout

```
Host Kernel:
    1 GB blocks

Guest RAM:
    2 MB blocks

Guest MMIO:
    4 KB pages (Device-nGnRnE)
```

Why this pattern works

- high VM density
- low TLB pressure
- fast migration
- safe device virtualization

****

****

21.11 — Pattern J: Deterministic Robotics System

****

****

Characteristics

- predictable timing
- deterministic memory access
- static mappings
- strict device isolation
- DMA-safe buffers

Memory Layout

```
Control Loop:
    Code: 2 MB block
    Data: 2 MB block

Sensors:
    MMIO: Device-nGnRnE

Actuators:
    MMIO: Device-nGnRnE

Shared Buffers:
    Normal WB, Inner Shareable
```

Why this pattern works

- deterministic latency
- safe device access
- predictable DMA behavior

****

****

21.12 — Pattern K: Minimalist Microkernel

****

****

Characteristics

- tiny kernel
- user servers
- message-passing architecture
- kernel mapped globally
- servers mapped via TTBR0

Memory Layout

```
Kernel:
    1 GB block

Servers:
    4 KB pages

MMIO:
    Device-nGnRnE
```

Why this pattern works

- minimal kernel footprint
- strong isolation
- simple MMU design

****

****

21.13 — Pattern L: High-Security System

****

****

Characteristics

- W^X enforced everywhere
- PXN/UXN enforced
- PAN enabled
- Stage-2 overrides guest permissions
- device memory isolated

Memory Layout

```
Code: executable, read-only
Data: writable, non-exec
Stack: writable, non-exec
MMIO: Device-nGnRnE
```

Why this pattern works

- strong exploit mitigation

- strict memory separation
- safe device access

****

****

21.14 — Pattern M: Real-Time Audio/Video System

****

****

Characteristics

- high bandwidth
- low latency
- DMA-heavy
- cache-optimized
- minimal fragmentation

Memory Layout

```
Audio buffers: Normal WB
Video buffers: Normal WB
MMIO: Device-nGnRnE
```

Why this pattern works

- high throughput
- low latency
- safe DMA

****

****

21.15 — Pattern N: Multi-Tenant Isolation

****

****

Characteristics

- per-tenant ASID ranges
- per-tenant VMID ranges
- strict Stage-2 isolation
- per-tenant device virtualization

Memory Layout

```
Tenant A:
    RAM: 2 MB blocks
Tenant B:
    RAM: 2 MB blocks
Tenant C:
    RAM: 2 MB blocks
```

Why this pattern works

- strong isolation
- scalable multi-tenant design

****

****

21.16 — Chapter Summary

****

****

This chapter covered **real MMU design patterns**, including:

- classic OS kernels
- hypervisors
- embedded systems
- safety-critical RTOS
- high-performance servers
- GPU/accelerator systems
- automotive ECUs
- secure enclaves
- cloud virtualization
- robotics
- microkernels
- high-security systems
- multimedia systems
- multi-tenant isolation